

L2: Max-flow Problem

Another problem that occurs naturally in networks that have a source node s and a target node t is to compute a “**flow**” from s to t .

The edge weights here represent the capacity of each edge, i.e., an edge of weight $w > 0$ cannot carry more than w flow units.

Additionally, edges cannot have a negative flow.

The total flow that arrives at a non-terminal node v has to be equal to the total flow that departs from v – in other words, flow is conserved.

The max-flow problem refers to computing the maximum amount of flow that can originate at s and terminate at t , subject to the capacity constraints and the flow conservation constraints.

The max-flow problem can be solved efficiently using the **Ford-Fulkerson algorithm**, as long as the capacities are rational numbers. In that case, the running time of the algorithm is $O(mF)$, where m is the number of edges and F is the maximum capacity of any edge.

The algorithm works by constructing a **residual network**, which shows at any point during the execution of the algorithm the residual capacity of each edge. In each iteration, the algorithm finds a path from s to t with some residual capacity (we can use BFS or DFS on the residual network to do that). Suppose that the minimum residual capacity among the edges of the chosen path is f . We add f on the flow of every edge (u,v) along that path, and decrease the capacity of those edges by f in the residual network. We also add f on the capacity of every reverse edge (v,u) of the residual network. The capacity of those reverse edges is necessary so that we can later reduce the flow along the edge (u,v) , if needed, by routing some flow on the edge (v,u) .



Food-for-thought:

Construct the residual network (also showing the flow on the reverse edges) for the example shown in the previous video.